Southern New Hampshire University

CS 405 Project Two Script Template

Gary Travis
4/19/2026

Presatition link
https://youtu.be/tmx6FgLRhO0

Complete this template by replacing the bracketed text with the relevant information.

Slide Number	Narrative
1	**Slide 1 – Title** For this presentation, I will walk through the Green Pace Secure Development Policy that I created for CS 405. My name is Gary Travis, and I will explain how this policy standardizes the way our team thinks about security in C and C++ code, data protection, and system architecture. The goal is to show how we can use principles, coding standards, and automated tools together to reduce vulnerabilities before they ever reach production.
2	**Slide 2 – Overview: Defense in Depth** On this slide, I introduce the overall purpose of the security policy. Green Pace builds custom applications that handle customer, financial, and internal data, so we need a consistent, repeatable way to write secure code. The policy defines ten core security principles, ten C and C++ coding standards, encryption rules, and Triple-A practices for authentication, authorization, and accounting. Taken together, these elements support a defense-in-depth approach, which means we do not rely on a single control or check. Instead, we layer protections in code, configuration, and operations so that if one control fails, others are still in place to prevent a complete compromise.

Slide Number	Narrative
3	**Slide 3 – Threats Matrix** The threats matrix summarizes how I see the relative risk of each coding standard. At the top right, in the “likely and high priority” area, I place the standards for validating data ranges, preventing SQL injection, and validating all external input at API and UI boundaries. These are STD-002, STD-004, and STD-010, and they represent issues that are both common and severe. In the “high priority but less likely” area, I include buffer and string handling, RAII and memory protection, exception safety, secure logging, and generic error messages. These are still serious, but often appear in more specific contexts. Assertions fall into a medium-severity, lower-priority category because misuse can cause problems, but it is less likely to be the first path an attacker uses. Overall, the matrix helps focus our effort on the most dangerous vulnerabilities first, while still addressing the others.
4	**Slide 4 – Ten Principles** This slide lists the ten core security principles that guide the policy: validate input data, heed compiler warnings, architect and design for security policies, keep it simple, default deny, least privilege, sanitize data sent to other systems, defense in depth, use effective quality assurance techniques, and adopt a secure coding standard. Underneath, I show how these principles map to specific coding standards. For example, Principle 1, validating input data, supports standards for data types, value ranges, and API boundary validation, which are STD-001, STD-002, and STD-010. Principle 7, sanitizing data sent to other systems, directly supports standards that prevent SQL injection, protect logs, and control error messages. Principle 10, adopting a secure coding standard, applies to all ten standards and ensures that the team has one consistent way to apply these ideas in C and C++.

Slide Number	Narrative
5	**Slide 5 – Coding Standards** On this slide, I summarize the ten C and C++ coding standards in the policy. Together they cover choosing safe data types, validating values and ranges, using bounded string operations, preventing SQL injection, managing memory with RAII and smart pointers, using assertions correctly, designing exception-safe code, logging securely, returning generic error messages, and validating inputs at UI and API boundaries. I explain that I prioritized these standards using the Severity, Likelihood, and Level values from the risk tables in the written policy. Critical issues that are likely and relatively inexpensive to fix, such as injection and input validation, are treated as top priority. Issues that are high severity but less likely or more expensive to fix are still important but come after the most urgent ones. This ranking gives the team a practical order in which to tackle technical debt.
6	**Slide 6 – Encryption Policies** Here I summarize the three encryption policies. Encryption at rest requires that all customer and sensitive internal data stored on disks, databases, and backups be protected with strong encryption and centrally managed keys, so stolen media cannot be read. Encryption in flight requires that all traffic carrying sensitive or authenticated data use TLS, such as HTTPS or secure database connections, to prevent eavesdropping and tampering on the network. Encryption in use focuses on how data is handled while it is being processed: applications should minimize how long secrets stay in memory, avoid writing them to logs or temporary files, and restrict access to processes that need them. Together, these policies

Slide Number	Narrative
	protect data throughout its life cycle: when it is stored, transmitted, and actively used.
7	**Slide 7 – Triple-A Policies** This slide describes the Triple-A framework: authentication, authorization, and accounting. Authentication requires unique user accounts and secure login mechanisms, and whenever possible multi-factor authentication, with all login attempts logged for auditing. Authorization uses role-based access control and least privilege so users only have the access they need, and any change to permissions or roles must be approved and logged. Accounting focuses on tracking what users and systems do after they have been authenticated and authorized. That includes logging key events such as logins, file access, database changes, and user management actions, and keeping logs protected and retained long enough for audits and incident response. These policies ensure we know who is accessing what, who is allowed to, and what they actually did.
8	**Slide 8 – Unit Testing Overview** In the unit testing section, I focus on one coding standard in depth: STD-010, which requires validating all external input at API and UI boundaries. The vulnerability I am testing is the use of unvalidated external input for counts, allocations, or array indexes, which can easily lead to buffer overflows, memory errors, or crashes. The target functions are `parse_positive_int` and `handle_request_compliant` from the policy document. My goal is to show that valid inputs are accepted and processed safely, while unsafe inputs are rejected before they can cause allocations or out-of-bounds writes. To do this, I use the Visual Studio C++ unit test framework along with static analysis tools such as Cppcheck and SonarQube, which confirm that the validation checks are actually present and enforce the policy.

Slide Number	Narrative
9	**Slide 9 – Test 1: Valid Numeric Input** The first test asks: "Does valid numeric input create the correct number of items?" In this test, the input string "10" is passed to `handle_request_compliant`. The expected behavior is that `parse_positive_int` returns true, the count is set to 10, and an items array of size 10 is allocated and filled. In the unit test, I call `handle_request_compliant("10")` and assert that no error message is printed to the console and that the function completes without throwing an exception or signaling a failure. If there is a side effect we can observe, such as a status return or a log entry, we verify that as well. When this test passes, it shows that the function handles normal, valid input correctly, which is just as important as handling bad input.
10	**Slide 10 – Test 2: Non-Numeric Input** The second test asks: "Does non-numeric input get rejected without allocation?" For this test, the input string "abc" is passed to `handle_request_compliant`. In this case, `parse_positive_int` should return false, and `handle_request_compliant` should print "Invalid item count." and return without creating an items array. The unit test calls `handle_request_compliant("abc")`, captures the console output, and asserts that the output contains the exact error message. We also confirm that there is no attempt to allocate or loop over an items array, for example by checking that the function returns early or that no observable side effect shows a successful operation. A passing result demonstrates that non-numeric input is safely rejected and that the vulnerability of using garbage values in allocations is removed.

Slide Number	Narrative
11	**Slide 11 – Test 3: Zero and Negative Values** The third test asks: "Are zero or negative counts rejected safely?" Here, I test two inputs, "0" and "-5", each passed separately into `handle_request_compliant`. The expectation is that `parse_positive_int` either fails or returns a value that fails the check `count <= 0 || count > MAX_ITEMS`, which causes the function to print "Invalid item count." and return. In the unit test, I call the function with both inputs, capture the console output, and assert that both calls produce the error message and exit early. When this test passes, it proves that invalid but syntactically numeric values do not slip through and cause zero-length allocations, negative sizes, or dangerous indexing.
12	**Slide 12 – Test 4: Excessively Large Number** The fourth test asks: "Does an excessively large number get blocked before overflow?" To simulate this, I use an input like "9999999999", which is larger than `MAX_ITEMS` and may also be larger than what an `int` can hold. The expected behavior is that `parse_positive_int` either detects overflow or returns a value that fails the range check against `MAX_ITEMS`. In both situations, `handle_request_compliant` should print "Invalid item count." and avoid allocating any memory. The unit test calls `handle_request_compliant("9999999999")`, captures the output, and asserts that the error message appears. When possible, I also run this test with AddressSanitizer enabled to confirm that no overflow or invalid allocation occurs at runtime. A passing test shows that the code properly defends against extreme values that attackers might use to force resource exhaustion or trigger integer overflows.

Slide Number	Narrative
13	**Slide 13 – Test 5: Maximum Allowed Value** The fifth test asks: "Does the maximum allowed value still work correctly?" In this case, I use the input "1000" when `MAX_ITEMS` is set to 1000. The expected behavior is that `parse_positive_int` succeeds, sets count to 1000, and that `handle_request_compliant` allocates an array of 1000 items, fills it, and deletes it safely. The unit test calls `handle_request_compliant("1000")`, asserts that no error message is printed, and confirms that the function returns successfully without memory errors. This can be reinforced by running under a memory checking tool like ASan. When the test passes, it shows that the code handles its upper boundary correctly: it neither rejects valid values nor overflows when they are at the configured limit.
14	**Slide 14 – Test 6: Leading Zeros** The sixth test asks: "Do values with leading zeros still parse correctly?" Here, the input is "007", which should still be interpreted as the positive number 7. The expected behavior is that `parse_positive_int` returns true and sets the count to 7, and `handle_request_compliant` allocates and fills an array of 7 items without error. The unit test calls `handle_request_compliant("007")`, asserts that no error message appears, and verifies that the function completes successfully. When this test passes, it demonstrates that the validation logic is strict about rejecting bad input but flexible enough to handle normal formatting variations that real users might provide.

Slide Number	Narrative
15	**Slide 15 – Automation Summary** This slide describes how automation supports the policy across the DevSecOps pipeline. In the Plan and Code stages, developers follow the ten C and C++ standards and run static analysis tools like Cppcheck and SonarQube as part of their normal workflow. In the Build and Test stages, the continuous integration pipeline compiles the code, runs the unit tests that I just demonstrated, and executes static analysis with the configured security rules, failing the build if critical or high-severity issues remain. Before Release and Deploy, a security gate checks that all tests passed and that there are no outstanding critical findings. Finally, during Operate and Monitor, log aggregation and monitoring tools use system, firewall, and anti-malware logs to detect violations and feed new test cases and policy updates back to development. This keeps enforcement consistent and continuous instead of relying only on one-time reviews.
16	**Slide 16 – Tools** On this slide, I focus on the specific tools and where they live in the DevSecOps pipeline. The pipeline stages we use are Plan and Code, Build and Test, Release and Deploy, and Operate and Monitor. In Code and Build/Test, we use static analyzers such as Cppcheck and SonarQube to detect missing input validation, unsafe memory usage, SQL injection patterns, and cases where sensitive data might be logged. For unit testing, we use the Visual Studio C++ test framework together with runtime tools like AddressSanitizer in debug builds to catch leaks, use-after-free errors, and buffer overflows under real test inputs. In Operate and Monitor, we rely on a log management platform combined with existing operating system, firewall, and anti-malware logs to detect suspicious activity and verify that our authentication, authorization, and encryption policies are being followed in production.

Slide Number	Narrative
17	**Slide 17 – Risks and Benefits** This slide explains the risks and benefits of applying these standards and tools. On the risk side, legacy C and C++ code and inconsistent practices can leave gaps in input validation, memory management, and error handling, which all increase the chance of critical vulnerabilities reaching production. On the solution side, applying the ten coding standards, enforcing encryption for data at rest and in flight, and following Triple-A policies for access control and auditing directly reduce these risks. Acting now comes with costs: refactoring code, writing tests, and adding automation take time and effort. However, the benefits are a lower likelihood of data breaches, outages, and expensive incident response. If we wait, vulnerabilities remain in the system, attackers have a larger surface area to exploit, and both technical and compliance risks increase. That is why I recommend addressing Critical and High issues, such as SQL injection, boundary input validation, and buffer handling, before tackling the less severe ones.
18	**Slide 18 – Recommendations** Here I present specific recommendations for improving the policy and its implementation. First, we should extend the policy to cover third-party libraries and dependencies, including regular updates and vulnerability scanning, because insecure libraries can bypass protections in our own code. Second, we should strengthen code reviews by adopting checklists aligned with the ten principles and coding standards, so reviewers consistently look for the same classes of issues, such as unvalidated input or unsafe logging. Third, we should increase automated test coverage for high-risk workflows, especially around authentication, changes to authorization, and

Slide Number	Narrative
	access to sensitive data, using both positive and negative test cases. Finally, we should keep the threats matrix and risk levels up to date based on new incidents, tool findings, and industry guidance, so that our priorities stay aligned with real-world threats.
19	**Slide 19 – Conclusions** In conclusion, Green Pace now has a standardized secure development policy that ties together ten guiding principles, ten concrete C and C++ coding standards, clear encryption requirements, and Triple-A practices. By embedding external tools and automated tests into the DevSecOps pipeline, we can consistently detect and prevent many common vulnerabilities before deployment, instead of relying solely on manual inspections. Moving forward, the key is to treat these standards as mandatory for new and modified code, to keep our tools and tests current, and to refine the policy based on lessons learned from production and from the broader security community. This approach gives the development team a clear, repeatable framework for protecting Green Pace's systems and data.
20	**Slide 20 – References** On the final slide, I list the key references that informed this policy and presentation. These include articles on secure coding practices, such as guidance on input validation, memory management, and defense in depth, as well as documentation for tools like Cppcheck, SonarQube, the Visual Studio C++ unit testing framework, and AddressSanitizer. I also reference the original Green Pace template and course materials, which define the required structure for principles, coding standards, and system policies. Together, these sources support the recommendations I have made and provide a foundation for the team to continue learning about secure coding and DevSecOps practices.

Southern New Hampshire University

Slide Number	Narrative